10-MINUTE PAPER REVIEW

# Heterogeneous Model Serving at Different Granularities

## ThunderServe and Hetis

**Yu-Jui Liang**
Advisor: Hsiang-Yun Cheng
National Taiwan University | Academia Sinica
28 Oct 2026

ThunderServe: Youhe Jiang et al., MLSys 2025
Hetis: Zizhao Mo et al., SC 2025

# Presentation Roadmap

28 OCT / PLAN

1. **ThunderServe + Hetis**
   10 minutes. Review and comparison of last week's papers. This deck.
2. **Helix**
   10 minutes. Concise report on a new paper.
3. **Tessera**
   10 minutes. Concise report on a new paper.
4. **Two HBF papers**
   Extra 10 minutes. Backup, not counted in the formal 30 minutes.

Formal report: three blocks, 30 minutes. This deck is the first block.

# Why Heterogeneous GPU Serving Needs Phase-Aware Scheduling

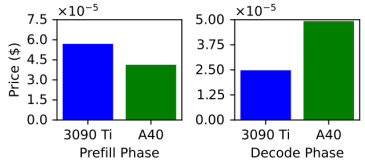



*Figure 1.* Prefill and decode prices for a single request with input and output lengths of 512 and 16 on 3090Ti and A40.

## Observation

- Prefill: compute-intensive.
- Decode: memory-bandwidth-intensive.

A40: 149.7 FP16 TFLOPS. 3090 Ti: 1008 GB/s. Table 1.

## Challenge

- Heterogeneous GPU resources.
- Limited and heterogeneous network bandwidth.
- KV cache transfer overhead.

**Research question**
How to jointly optimize GPU allocation, phase placement, and communication?

**The better GPU can differ by phase. KV transfer makes a FLOPS-only or bandwidth-only split insufficient.**

Source: ThunderServe Sec. 1, Fig. 1, Table 1. Prices in the figure are per request.

# Joint Optimization of GPU Placement, Parallelism, and Routing

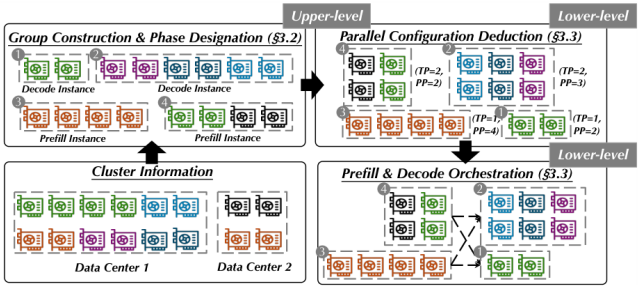



*Figure 3.* Workflow of our scheduling algorithm.

**UPPER-LEVEL — TABU SEARCH**

- GPU group construction.
- Prefill / decode phase designation.

**LOWER-LEVEL — CONFIGURATION**

- TP / PP configuration.
- Prefill / decode request routing.
- Network-aware KV cache transfer.

Additional mechanisms: KV cache compression and lightweight rescheduling.

**Upper level assigns groups and phases. Lower level sets parallelism and routing. Together they score one deployment plan.**

Source: ThunderServe Sec. 3.2–3.4, Fig. 3.

# Does Phase-Aware Scheduling Improve Serving Efficiency?

**EXPERIMENT 1 — TABLE 3**

**Workload-aware deployment**

Coding: **8 prefill / 4 decode**

Conversation: **4 prefill / 8 decode**

LLaMA-30B, same 32-GPU cloud pool. Coding has longer prompts; conversation has longer outputs.

**EXPERIMENT 2 — FIGURE 9**

| | Coding | Conv. |
|---|---|---|
| vs. HexGen | 1.5× | 1.3× |
| vs. DistServe | 1.5× | 2.1× |

HexGen: heterogeneous cloud GPUs.
DistServe: homogeneous 8×A100, similar price. Not the same machines.

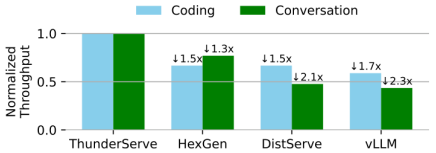



*Figure 9.* Throughput scaled by ThunderServe.

vLLM, the fourth group, is also in-house. HexGen and DistServe are different hardware setups.

**Workload-aware placement improves overall serving throughput under heterogeneous resource constraints.**

Source: ThunderServe Sec. 5.1–5.3, Fig. 9, Table 3. Relative throughput is ThunderServe over each baseline.

# Phase-Aware Deployment Improves Cost Efficiency

1. JOINT OPTIMIZATION

## One deployment plan

GPU grouping, phase designation, TP/PP, and request routing.

2. PERFORMANCE

## 1.7× average, up to 2.1×

Overall throughput claim in the paper. Not one bar of Figure 9.

3. DYNAMIC ADAPTATION

## Full 157 s, lightweight 13 s

Table 4 overall time. The gap includes model reload, not search time alone.

Table 4 breakdown. Full: rescheduling $54(\pm5)$ s + reload $103(\pm10)$ s = $157(\pm13)$ s. Lightweight: rescheduling $13(\pm2)$ s + reload 0 s = $13(\pm2)$ s.

**ThunderServe demonstrates that heterogeneous GPU serving benefits from jointly optimizing inference phases, hardware allocation, and communication.**

ThunderServe focuses on phase-level resource allocation; finer-grained workload partitioning is another possible direction.

Source: Abstract and Sec. 6 for the 1.7×/2.1× claim. Sec. 5.4 and Table 4 for rescheduling time.

# Hetis: Why Static Parallelism Wastes GPU Resources

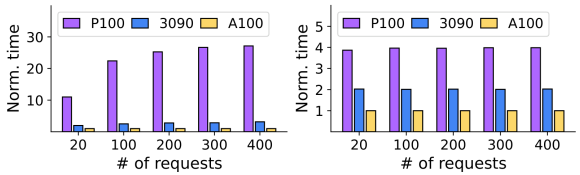



**Figure 2: The execution time of the decoding MLP and Attention of one layer in Llama-70B across different GPUs. Sequence length of each request is set to 1000.**

## Why one parallelism for every module?

- MLP and decode attention do not show the same GPU gap.
- A low-end GPU can drag dense computation.
- A static split can also leave KV-cache space idle.

Fig. 2 is one Llama-70B decode layer, sequence length 1000. Not end-to-end latency, and not a bandwidth chart. The paper asks for module-aware, fine-grained, dynamic parallelism.

**GPU performance heterogeneity is module-dependent; static model partitioning cannot efficiently balance both computation and memory.**

Source: Hetis Sec. 2.2–2.4, Figs. 1–2. Text: P100 lags A100 by up to 40.4× on average in decode MLP.

# Hetis: Selective Primary Workers and Dynamic Head-wise Attention

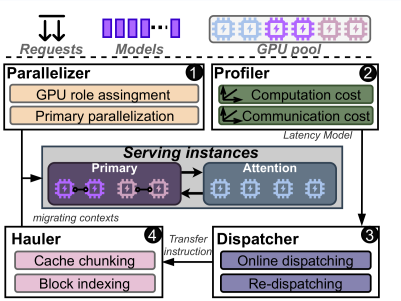



Figure 3: The system architecture of Hetis.

**PRIMARY WORKERS**

Parallelizer searches dense-compute plus communication cost. They jointly hold the model with TP/PP, and DP when needed. They run prefill, decode dense modules, and heads that stay local.

**ATTENTION WORKERS**

GPUs that add little to dense compute. They run some decode attention and its KV cache, without a second full copy of the weights. Default exclusion threshold $\Delta = 0.05$. Selection is cost, not a FLOPS or bandwidth ranking.

**PER-LAYER DECODE TRAFFIC**

Primary computes QKV. For an offloaded head it sends that token's Q/K/V; the worker attends with its stored KV cache and returns the output. Full history is not resent each step. GQA keeps a query group with its KV heads.

**Hetis concentrates dense computation on suitable GPUs while dynamically distributing decode attention at head granularity.**

Source: Hetis Sec. 3–5, Fig. 3. Fig. 3 also names Profiler, Dispatcher, and Hauler.

# Hetis: Does Head-wise Parallelism Improve Serving Performance?

**Q1 — END-TO-END SERVING**

Fig. 10 y-axis is latency (s/token), not throughput.

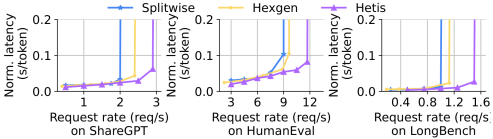



**Figure 10: The normalized end-to-end latency across different datasets under Llama-70B.**

Up to 2.25× throughput vs. Splitwise, and 1.33× vs. Hexgen. Maxima across settings, not averages. On this Llama-70B run, A100 and 3090 were primaries and P100s were attention workers. A search result, not a ranking.

**Q2 — KV CACHE**

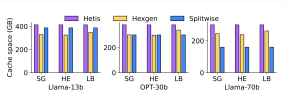



**Figure 11: The maximum available KV Cache space during inference across models and datasets.**

Up to 1.87× available KV cache versus the tested baselines.

**Fine-grained attention placement improves serving capacity by reducing compute imbalance and KV cache underutilization.**

Source: Hetis Sec. 7.1–7.3, Figs. 8–13. The 1.49× figure is decode-attention module latency, not end-to-end latency.

# Hetis: Main Contributions and Takeaways

1

## Module-aware workers

Assign GPUs by what each module costs, not by one static split of the whole model.

2

## Head-level attention

Offload decode attention by heads. Unselected heads stay on the primary.

3

## Online adaptation

Dispatcher reallocates heads. Hauler migrates KV cache at head granularity.

**Implementation: vLLM extension.**

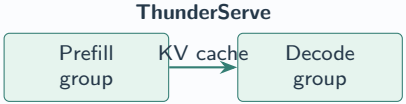
Head-level KV-cache blocks, custom CUDA kernels, NCCL peer-to-peer, and reuse of PagedAttention. Base framework, not a Hetis repository: `https://github.com/vllm-project/vllm`

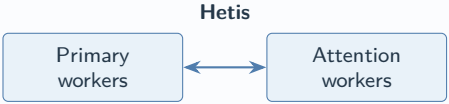
End-to-end results use the real GPU cluster. Sec. 7.4 adds a large-scale simulation of parallelizer search cost.

**Hetis improves heterogeneous GPU utilization through module-aware, fine-grained, and dynamic parallelism.**

Source: Hetis Sec. 6 and Sec. 9 for the system; Sec. 7.1–7.2 for the cluster results; Sec. 7.4 for the search simulation.

# ThunderServe vs. Hetis: Different Granularities of GPU Specialization




| | ThunderServe | Hetis |
|---|---|---|
| **Objective** | Cost-efficient serving under workload and network limits | Less compute imbalance and better memory use |
| **Granularity** | Prefill / decode phases | Dense modules / attention heads |
| **Weights** | Each phase replica hosts weights; TP/PP allowed | Primaries hold the model. Attention workers do not |
| **Communication** | KV cache between prefill and decode replicas | Current-token Q/K/V and outputs for offloaded heads |
| **Scheduling** | Grouping, phase, TP/PP, routing, lightweight rescheduling | Primary selection, online head dispatch, KV migration |

**ThunderServe optimizes phase-level deployment; Hetis optimizes module-aware execution and head-level attention placement.**

Source: Diagrams are conceptual, not paper figures. Do not rank the papers by peak throughput: hardware, workloads, and baselines differ.

10-MINUTE PAPER REVIEW

# Thank you

ThunderServe and Hetis

**Yu-Jui Liang**

28 Oct 2026